

INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE OCCIDENTE

APRENDIZAJE PROFUNDO



PROYECTO 3 – TRANSFORMERS + ATTENTION

Presenta:

Luis Eduardo Jiménez del Muro – 746484 – eduardo.jimenez@iteso.mx

Santiago Reyes Castillo – 745826 – santiago.reyes@iteso.mx

Santiago Montes de Oca – 745377 – smontesdeoca.-@iteso.mx

Profesor: Iván Reyes Amezcua

Fecha: 12/05/2026

1. Introducción	3
2. Objetivo	3
3. Marco Teórico.....	4
3.1 LLMs, Modelos de lenguaje de gran tamaño	4
Arquitectura Transformer.....	4
Pretraining	5
Instrucción tuning	5
El modelo Meta “Llama 3.3 70B Instruct”	6
Comparación de modelos.....	7
3.2 AI Agents y el patrón ReAct	8
3.3 Prompt Engineering.....	10
3.4 Llama 3.3 70B Instruct.....	11
3.5 Sentiment Analysis con FinBERT	12
Arquitectura BERT	13
Transfer Learning y Fine-Tuning	14
4. Metodología	15
4.1 Funcionamiento del agente	15
4.3 System prompt	16
Definición del Rol del Agente	17
Reglas Obligatorias (Strict Rules).....	17
Data Source Routing	18
Formato de Respuesta	18
Definición de Herramientas (Available Tools)	18
Ejemplos de Uso (Few-Shot Prompting)	20
Ejemplos Incorrectos y Restricciones.....	20
4.4 Herramientas	20
5. Resultados.....	22
5.2 Areas de mejora	24
6. Conclusiones	25
7. Referencias:	26
8. Repositorio en GitHub	27
9. Demostración en Hugging Face	27
10. Video demostrativo	27

1. Introducción

Los Modelos de Lenguaje de Gran Tamaño (LLMs) han transformado de manera significativa la forma en que los sistemas de inteligencia artificial interactúan con el lenguaje humano (GeeksforGeeks, 2026). Gracias a arquitecturas como el Transformer y técnicas de entrenamiento como el instruction tuning, estos modelos son capaces de comprender instrucciones complejas, razonar sobre información y generar respuestas contextualmente coherentes (GeeksforGeeks, 2026). Sin embargo, uno de los problemas más relevantes de los LLMs es su tendencia a "alucinar", es decir, generar información falsa con aparente confianza, lo cual representa un riesgo crítico en dominios sensibles como el financiero (Siddiqui, 2025).

En este contexto, los AI Agents ofrecen una solución prometedora: en lugar de depender exclusivamente del conocimiento interno del modelo, estos sistemas combinan razonamiento iterativo con el uso de herramientas externas para obtener datos reales y verificables. Mediante el patrón ReAct (Reasoning and Acting), un agente puede analizar una pregunta, determinar qué información necesita, consultarla a través de APIs o librerías especializadas, y construir una respuesta fundamentada en datos concretos.

El presente proyecto desarrolla un agente financiero conversacional basado en el modelo Meta Llama 3.3 70B Instruct, capaz de responder preguntas sobre empresas, decisiones de inversión, indicadores macroeconómicos mexicanos, análisis de sentimiento de mercado, entre otros. El agente integra herramientas de datos financieros en tiempo real, indicadores de Banxico y análisis de sentimiento mediante FinBERT, todo ello orquestado a través de un sistema de prompt engineering cuidadosamente diseñado para controlar el comportamiento del modelo sin necesidad de reentrenarlo.

2. Objetivo

El objetivo de este proyecto es diseñar e implementar un agente de inteligencia artificial financiero basado en el modelo Llama 3.3 70B Instruct, que sea capaz de responder preguntas sobre precios de acciones, indicadores macroeconómicos mexicanos, optimización de portafolios y análisis de sentimiento de mercado, utilizando exclusivamente datos obtenidos mediante herramientas externas. A través del patrón ReAct y técnicas de prompt engineering, se busca demostrar que es posible construir un

sistema conversacional confiable y verificable en el dominio financiero, eliminando la dependencia del conocimiento interno del modelo y reduciendo así el riesgo de alucinaciones en un entorno donde la precisión de la información es crítica.

3. Marco Teórico

3.1 LLMs, Modelos de lenguaje de gran tamaño

Los LLMs (por sus siglas en inglés Large Language Models) son sistemas de inteligencia artificial diseñados para comprender y generar lenguaje natural. Estos modelos son entrenados utilizando enormes cantidades de texto provenientes de libros, artículos, páginas web y otras fuentes digitales, lo que les permite aprender patrones lingüísticos, estructuras gramaticales, relaciones semánticas y contexto conversacional (GeeksforGeeks, 2026).

Los LLMs funcionan prediciendo la siguiente palabra o fragmento de texto más probable dentro de una secuencia. Aunque este principio parece sencillo, el entrenamiento sobre grandes volúmenes de información y el uso de arquitecturas avanzadas permiten que estos modelos desarrollen capacidades complejas, como responder preguntas, resumir textos, traducir idiomas, generar código y mantener conversaciones coherentes (GeeksforGeeks, 2026).

Arquitectura Transformer

La mayoría de los LLMs modernos están basados en la arquitectura Transformer, presentada en 2017 por investigadores de Google en el artículo Attention Is All You Need. Esta arquitectura revolucionó el procesamiento de lenguaje natural debido a su capacidad para procesar grandes cantidades de texto de manera eficiente y paralela (Vaswani et al., 2017) .

El componente principal del Transformer es el mecanismo de self-attention o autoatención. Este mecanismo permite que el modelo determine qué palabras dentro de una oración son más relevantes entre sí, incluso cuando se encuentran separadas por grandes distancias dentro del texto. Gracias a ello, el modelo puede comprender mejor el contexto y las relaciones semánticas (GeeksforGeeks, 2026).

Pretraining

El pretraining o preentrenamiento es la etapa inicial en la que un LLM aprende patrones generales del lenguaje. Durante esta fase, el modelo es entrenado utilizando enormes conjuntos de datos textuales sin supervisión explícita (Nieto, 2025).

El objetivo principal consiste en que el modelo aprenda a predecir el siguiente token dentro de una secuencia de texto. A través de millones o miles de millones de ejemplos, el sistema desarrolla conocimiento estadístico sobre gramática, sintaxis, contexto y relaciones conceptuales.

Esta etapa requiere una gran capacidad computacional, generalmente utilizando clusters de GPUs o TPUs especializadas para entrenamiento de redes neuronales profundas. El tamaño del modelo y la cantidad de datos utilizados tienen un impacto importante en las capacidades finales del sistema. Esta es una etapa que simplemente no llegamos a ver dentro de el curso debido a la cantidad de recursos que se necesitan para preentrenar un modelo, y adicionalmente con la cantidad de modelos ya entrenados que existen en la web, es mucho mas practico simplemente conseguir estos modelos ya preentrenados (Nieto, 2025).

El preentrenamiento proporciona al modelo una comprensión general del lenguaje, pero no necesariamente garantiza que pueda seguir instrucciones humanas de manera precisa o segura. Por ello, posteriormente se realizan etapas adicionales de ajuste para combatir justo estos puntos de interes.

Instrucción tuning

Hablando de ajustar un modelo, podemos hablar de dos métodos: lo es el “fine-tuning” y el “prompt engineering”. Ambos cumplen la misma meta, pero con métodos completamente diferentes (Bergmann, 2025). El fine-tuning básicamente toma el modelo ya preentrenado y le agrega etapas de entrenamiento para reforzar su capacidad de aprender ciertos patrones en los datos. El prompt engineering, que llegaremos a explicar con más detalle posteriormente, se enfoca en modificar no los pesos del modelo, como en el fine-tuning, sino en colocar al modelo en un rol en el que llegue a entender lo que el usuario quiere (Bergmann, 2025).

Debido al modelo que se utiliza en el proyecto, “llama-3.3-70b-instruct”, el cual consultamos a través de una API, se volvió imposible implementar el método de fine-

tuning. Por ende, la manera en que llegamos a personalizar el modelo fue a través de prompt engineering.

El modelo Meta “Llama 3.3 70B Instruct”

El modelo “Llama 3.3 70B Instruct” pertenece a la familia de modelos Llama desarrollada por Meta. Se trata de un LLM orientado a tareas conversacionales e instrucciones, optimizado para ofrecer respuestas útiles y contextualmente coherentes.

El término “70B” indica que el modelo posee aproximadamente 70 mil millones de parámetros, los cuales representan los valores internos aprendidos durante el entrenamiento. Un mayor número de parámetros suele permitir una mejor capacidad para modelar relaciones complejas del lenguaje, aunque también implica mayores requisitos computacionales.

La versión “Instruct” del modelo ha sido específicamente ajustada mediante instruction tuning para responder instrucciones humanas de manera más efectiva. Esto lo hace especialmente útil para aplicaciones como:

- Asistentes virtuales.
- Generación de texto.
- Resumen de información.
- Traducción.
- Soporte técnico.
- Generación y análisis de código.

Una de las principales ventajas de la familia Llama es su enfoque relativamente abierto para investigación y desarrollo, permitiendo que universidades, empresas y desarrolladores puedan implementar soluciones personalizadas utilizando estos modelos.

Además, Llama 3.3 incorpora mejoras en comprensión contextual, razonamiento y eficiencia respecto a versiones anteriores, convirtiéndose en una opción competitiva dentro del área de inteligencia artificial generativa (*Hugging Face*, 2024).

Comparación de modelos

Durante las prácticas de clases se trabajó con distintos modelos de lenguaje, entre ellos el modelo “openai-community/openai-gpt” y el modelo “Qwen/Qwen2.5-1.5B-Instruct”. Estos modelos presentan diferencias importantes en arquitectura, capacidad y número de parámetros.

El modelo OpenAI “openai-gpt” corresponde a una generación temprana de modelos GPT. Fue presentado en 2018 y contaba aproximadamente con 117 millones de parámetros, siendo considerablemente más pequeño que los modelos actuales. A pesar de sus limitaciones comparado con tecnologías modernas, este modelo representó un avance importante en el uso de pretraining basado en Transformers para tareas de procesamiento de lenguaje natural.

Por otro lado, el modelo Alibaba Cloud “Qwen2.5-1.5B-Instruct” pertenece a una generación mucho más reciente de LLMs. Este modelo cuenta con aproximadamente 1.5 mil millones de parámetros, lo cual representa más de diez veces el tamaño del modelo GPT original utilizado en clase. Además, incorpora técnicas modernas de instruction tuning, permitiéndole seguir instrucciones y mantener conversaciones de manera mucho más eficiente.

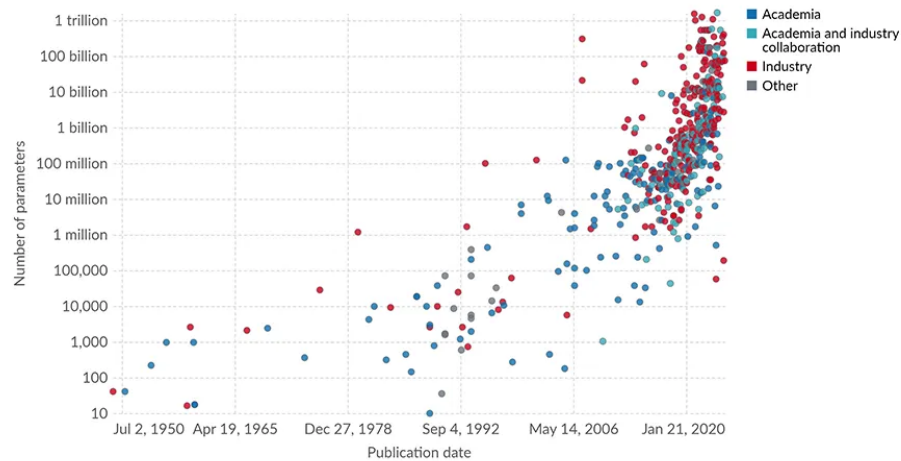
Finalmente, el modelo Meta “Llama 3.3 70B Instruct” posee aproximadamente 70 mil millones de parámetros, colocándolo en una categoría completamente distinta en términos de capacidad computacional, comprensión contextual y razonamiento. Según la documentación oficial de Meta, este modelo fue optimizado para tareas conversacionales y logra resultados competitivos en benchmarks de razonamiento, programación y matemáticas.

Modelo	Parámetros aproximados	Tipo
openai-community/openai-gpt	117 millones	Base / Pretrained
Qwen2.5-1.5B-Instruct	1.54 mil millones	Instruction tuned

Llama 3.3 70B Instruct	70 mil millones	Instruction tuned
------------------------	-----------------	-------------------

Parameters in notable artificial intelligence systems

Parameters are variables in an AI system whose values are adjusted during training to establish how input data gets transformed into the desired output; for example, the connection weights in an artificial neural network.



Data source: Epoch (2024)

OurWorldinData.org/artificial-intelligence | CC BY

Note: Parameters are estimated based on published results in the AI literature and come with some uncertainty. The authors expect the estimates to be correct within a factor of 10.

Figura 1, llega a demostrar el número de parámetros al pasar el tiempo, termina en 2024.

<https://ayarlabs.com/glossary/parameters-ai-ml/>

3.2 AI Agents y el patrón ReAct

Un agente de inteligencia artificial (AI Agent) es un sistema diseñado para percibir información de su entorno, razonar sobre ella y ejecutar acciones con el objetivo de cumplir una tarea específica. A diferencia de un modelo de lenguaje tradicional que únicamente responde una solicitud directa, un agente puede seguir procesos iterativos donde analiza información, toma decisiones y utiliza herramientas externas para resolver problemas más complejos (Chawla, 2026).

Primero, el agente piensa cual es la siguiente accione que debe hacer, despues utiliza una herramienta, como llamar a una tool, para acercarse mas al resultado final, observa el proceso que lleva y decide si ya entregar la respuesta final, o en su defecto, necesita

mas pasos para llegar a la respuesta final. A esto tambien se le llama razonamiento multi-step (multi-step reasoning).

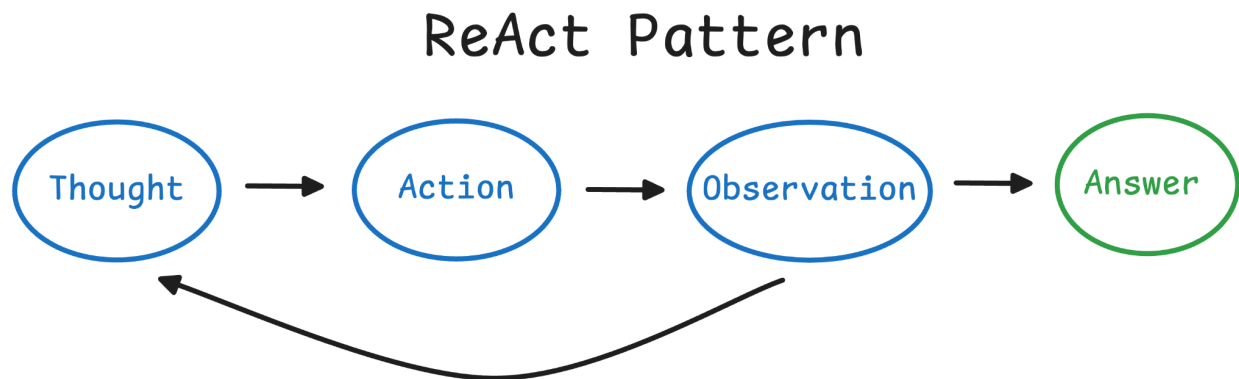


Imagen 2 representado el cómo se llega a desarrollar el patrón de React.

<https://www.dailydoseofds.com/ai-agents-crash-course-part-10-with-implementation/>

Uno de los enfoques más utilizados para este tipo de sistemas es el patrón ReAct (Reasoning and Acting), propuesto para combinar razonamiento textual con ejecución de acciones (Chawla, 2026). Este paradigma permite que el modelo no solo genere respuestas, sino que también interactúe dinámicamente con herramientas externas durante el proceso de resolución. De esta manera podemos evitar uno de los grandes problemas que tiene la IA de alucinar resultados, o dar información falsa con mucha confianza (Bergmann, 2026).

El patrón ReAct se basa en alternar pasos de razonamiento y acciones. El modelo puede:

- Analizar el problema.
- Determinar qué información necesita.
- Llamar herramientas externas.
- Evaluar los resultados obtenidos.
- Continuar razonando hasta producir una respuesta final.

Por ejemplo, un agente puede recibir una pregunta compleja, decidir buscar información utilizando una API externa, analizar los datos obtenidos y posteriormente generar una respuesta basada en esos resultados (Bergmann, 2026). Este enfoque resulta

especialmente útil en tareas donde el conocimiento interno del modelo no es suficiente o donde se requiere interacción con sistemas externos, como:

- Motores de búsqueda.
- Bases de datos.
- Calculadoras.
- APIs.
- Herramientas de análisis de código.

3.3 Prompt Engineering

El Prompt Engineering es el proceso de diseñar y estructurar instrucciones para modelos de lenguaje con el objetivo de controlar su comportamiento y mejorar la calidad de sus respuestas (Gadesha, 2026). Debido a que los LLMs generan resultados basados en patrones aprendidos durante el entrenamiento, la forma en que se redacta un prompt tiene un impacto mayor en el desempeño del sistema (Gadesha, 2026). Es importante redactar las instrucciones y conocimientos generales que se utilizan para hacer un System prompt apropiado, existe muchísima volatilidad en cuanto a lo que el modelo puede hacer, un ejemplo bueno para describir este fenómeno sería cuando al modelo de grok se le cambiaron unas cuantas líneas de su system prompt y terminó con opiniones altamente controversiales: "The response should not shy away from making claims which are politically incorrect, as long as they are well substantiated", este reporte no es 100% seguro, pero se reporta que el agregar esta línea es la razón que el modelo puede llegar a cometer tantos errores (Field, 2025).

En este proyecto, el sistema sigue un enfoque prompt-first, donde gran parte del comportamiento del agente es definido mediante prompts estructurados en lugar de modificaciones internas al modelo. Esto permite personalizar tareas y respuestas sin necesidad de reentrenar el LLM.

Uno de los componentes más importantes es el system prompt. Este prompt inicial establece reglas, restricciones y objetivos generales que el modelo debe seguir durante toda la conversación. El system prompt puede definir:

- El rol del asistente.
- El formato esperado de las respuestas.
- Restricciones de comportamiento.
- Uso de herramientas externas.

Gracias a ello, es posible controlar parcialmente la personalidad y funcionamiento del modelo dentro de un sistema conversacional. Dentro del system prompt, se utilizan técnicas para estructurar lo que se necesita o pide un modelo, y con ello, utilizando “few-shot” examples. Esta estrategia consiste en proporcionar ejemplos previos dentro del prompt para demostrar cómo debe responder el modelo. En lugar de únicamente describir una tarea, se muestran ejemplos de entradas y salidas esperadas, ayudando al LLM a identificar patrones de comportamiento deseados (Gadesha, 2025).

El proyecto también utiliza restricciones de formato de salida (output format constraints). Estas restricciones obligan al modelo a generar respuestas siguiendo una estructura específica, como JSON, listas o formatos predefinidos. Esto resulta especialmente útil cuando las respuestas deben ser procesadas automáticamente por otros componentes del sistema (Gadesha, 2025).

Adicionalmente, se emplean conceptos relacionados con chain-of-thought prompting. Esta técnica busca que el modelo genere razonamientos intermedios antes de producir una respuesta final. Al dividir el problema en pasos lógicos, el LLM puede mejorar su desempeño en tareas de razonamiento complejo, matemáticas o toma de decisiones (Google Cloud, 2026).

3.4 Llama 3.3 70B Instruct

El modelo de lenguaje grande (LLM) Meta Llama 3.3 es un modelo generativo ajustado mediante instrucciones en su versión de 70B parámetros (entrada de texto/salida de texto). El modelo Llama 3.3 ajustado mediante instrucciones y enfocado únicamente en texto está optimizado para casos de uso de diálogo multilingüe y supera a muchos de los modelos de chat de código abierto y cerrado disponibles en benchmarks comunes de la industria (Hugging Face, 2024).

Llama 3.3 es un modelo de lenguaje autorregresivo que utiliza una arquitectura transformer optimizada. Las versiones ajustadas utilizan fine-tuning supervisado (SFT) y aprendizaje por refuerzo con retroalimentación humana (RLHF) para alinearse con las preferencias humanas en términos de utilidad y seguridad.

Datos de entrenamiento	Una nueva mezcla de datos públicos disponibles en línea
Parámetros	70B
Modalidades de entrada	Texto multilingüe
Modalidades de salida	Texto multilingüe y código
Longitud de contexto	128k
GQA	Sí
Cantidad de tokens	15T+
Corte de conocimiento	Diciembre de 2023
Idiomas soportados	Inglés, alemán, francés, italiano, portugués, hindi, español y tailandés.
Fecha de lanzamiento del modelo	6 de diciembre de 2024

Los conteos de tokens se refieren únicamente a los datos de preentrenamiento. Todas las versiones del modelo utilizan Grouped-Query Attention (GQA) para mejorar la escalabilidad durante la inferencia (Hugging Face, 2024).

3.5 Sentiment Analysis con FinBERT

El análisis de sentimientos (Sentiment Analysis) es una técnica de procesamiento de lenguaje natural (Natural Language Processing, NLP) utilizada para identificar y clasificar

emociones, opiniones o posturas presentes dentro de un texto. En aplicaciones financieras, esta técnica permite analizar noticias, reportes y publicaciones relacionadas con empresas o mercados para determinar si el tono general es positivo, negativo o neutral.

En el proyecto desarrollado, el análisis de sentimiento es realizado mediante la herramienta *get_news_sentiment*, la cual obtiene noticias recientes relacionadas con una acción bursátil específica y utiliza el modelo FinBERT para calcular un puntaje compuesto de sentimiento financiero (ProsusAI, n.d.).

Arquitectura BERT

FinBERT está basado en la arquitectura BERT (Bidirectional Encoder Representations from Transformers), desarrollada por Google en 2018. BERT representa uno de los avances más importantes dentro del procesamiento de lenguaje natural debido a su capacidad para comprender contexto bidireccional.

A diferencia de modelos anteriores que procesaban texto únicamente de izquierda a derecha o de derecha a izquierda, BERT analiza simultáneamente el contexto completo de una oración. Esto permite interpretar mejor el significado de las palabras dependiendo de su contexto específico.

La arquitectura de BERT está basada en Transformers tipo encoder, utilizando mecanismos de autoatención (self-attention) para relacionar palabras entre sí dentro de una secuencia de texto. Gracias a ello, el modelo puede comprender relaciones semánticas complejas y capturar mejor el significado contextual de frases y documentos.

Durante su entrenamiento original, BERT utiliza tareas como:

- Masked Language Modeling (MLM): el modelo intenta predecir palabras ocultas dentro de una oración.
- Next Sentence Prediction (NSP): el modelo aprende relaciones entre pares de oraciones.
- Estas tareas permiten que el modelo desarrolle representaciones profundas del lenguaje que posteriormente pueden adaptarse a tareas específicas.

Transfer Learning y Fine-Tuning

Uno de los conceptos más importantes utilizados en FinBERT es el transfer learning. Este enfoque consiste en reutilizar un modelo previamente entrenado sobre grandes cantidades de texto y posteriormente ajustarlo (fine-tuning) para una tarea especializada (ProsusAI, n.d.).

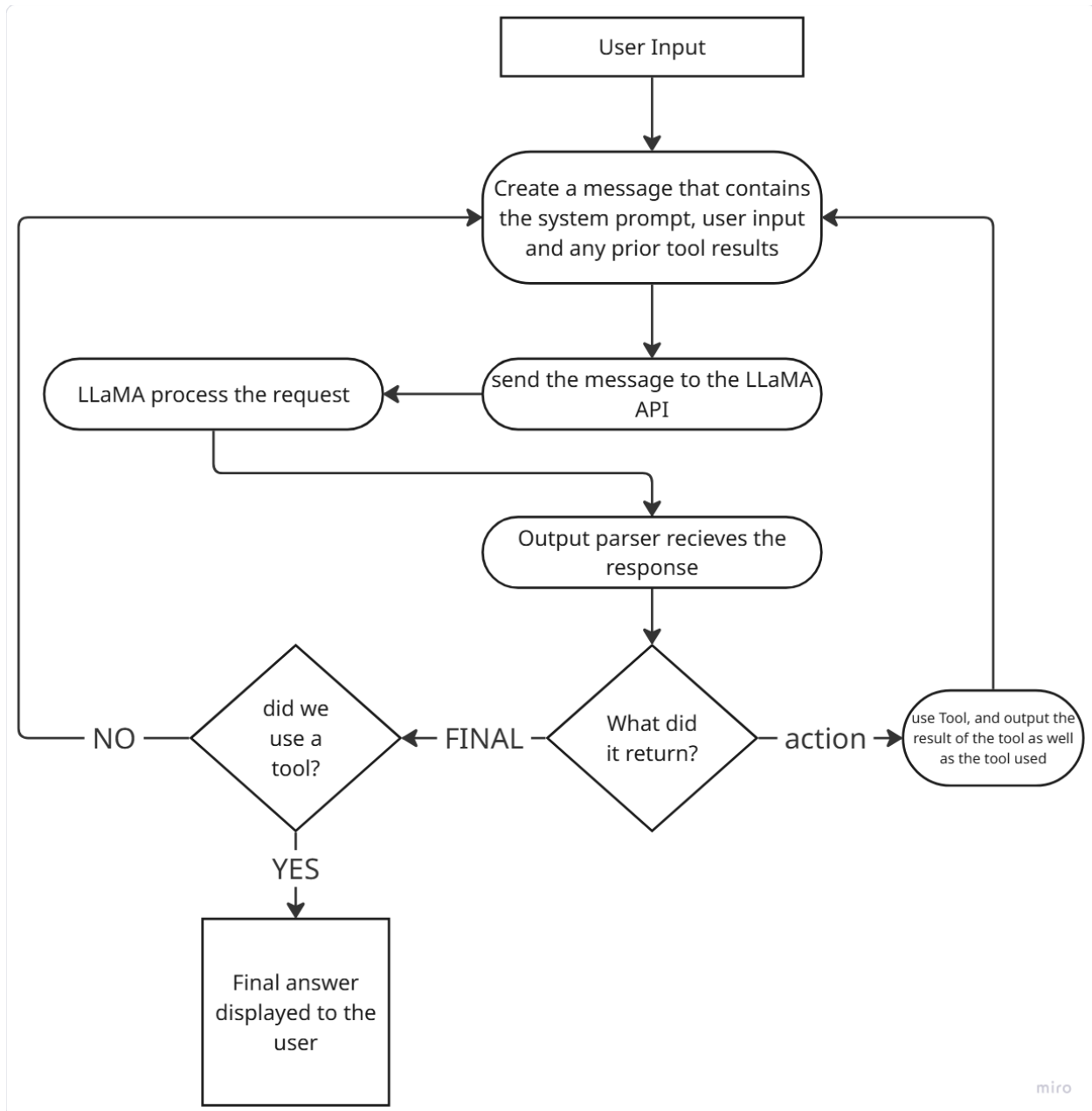
Entrenar un modelo Transformer desde cero requiere enormes cantidades de datos y recursos computacionales. Por ello, resulta más eficiente utilizar modelos preentrenados y adaptarlos a dominios concretos. En el caso de FinBERT, el modelo toma como base BERT y posteriormente es ajustado utilizando textos financieros, incluyendo:

- Noticias bursátiles.
- Reportes financieros.
- Información de mercados.
- Análisis económicos.

Gracias a este proceso de fine-tuning, el modelo aprende terminología y patrones específicos del lenguaje financiero, permitiendo una clasificación de sentimiento mucho más precisa que un modelo generalista. El modelo utilizado en el proyecto corresponde a FinBERT, desarrollado específicamente para clasificación de sentimiento financiero (ProsusAI, n.d.).

4. Metodología

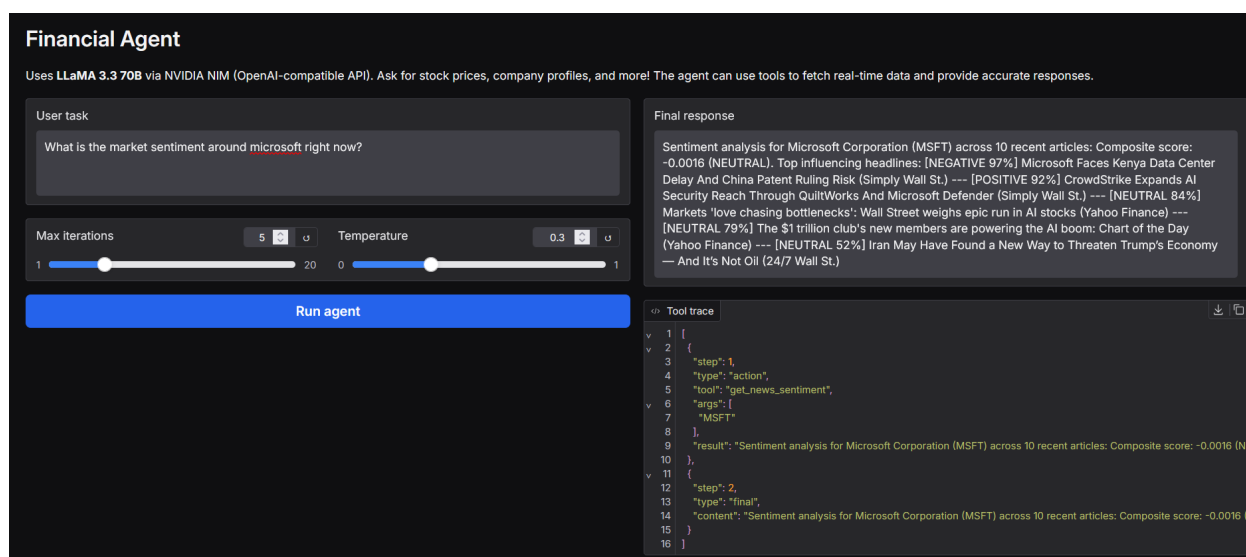
4.1 Funcionamiento del agente



El usuario escribe una pregunta en lenguaje natural dentro de la interfaz web de Gradio (por ejemplo: “¿Cuál es el precio de las acciones de Apple?”). El ciclo del agente toma el control. Este funciona como el orquestador: mantiene un historial continuo de la conversación y llama repetidamente al LLM hasta obtener una respuesta final (hasta un máximo de n cantidad de pasos definidos).

El LLM lee el historial completo (el system prompt, la pregunta del usuario y cualquier resultado previo de herramientas) y escribe exactamente una línea de respuesta: ya sea ACTION: nombre_herramienta(argumentos) o FINAL: <respuesta>. Nunca tiene permitido responder únicamente desde memoria. El parser, posteriormente lee esa línea y decide qué camino seguir. Si el formato es incorrecto, el agente envía un error de regreso y vuelve a intentarlo.

Si la respuesta es una ACTION, el agente llama a la herramienta correspondiente: una función en Python que obtiene datos reales (desde yfinance, la API de Banxico o FinBERT). El resultado se agrega al historial de mensajes como una “observación” y se envía nuevamente al LLM para el siguiente paso. Si la respuesta es FINAL, el agente primero verifica que al menos una herramienta haya sido utilizada previamente. Si ninguna herramienta fue ejecutada, bloquea la respuesta y fuerza otro paso (esto evita que el LLM invente información). Si sí se utilizó una herramienta, la respuesta es aceptada y devuelta. La respuesta final y el registro completo de herramientas utilizadas (cada paso, nombre de la herramienta, argumentos y resultado) se muestran lado a lado en la interfaz de usuario de Gradio. La interfaz se ve de la siguiente manera:



4.3 System prompt

El system prompt del agente fue diseñado de manera modular, separando instrucciones, reglas y definiciones de herramientas en bloques claramente delimitados. Esta estructura permite controlar el comportamiento del modelo de lenguaje y asegurar que todas las respuestas utilicen información obtenida mediante herramientas externas en lugar de memoria interna del modelo.

Definición del Rol del Agente

La primera sección consiste en una descripción corta y directa del propósito del agente. En este caso, el modelo es definido como un “financial data agent”, especializado en responder preguntas sobre empresas e indicadores macroeconómicos. Esta sección funciona como una instrucción de alto nivel que establece el contexto general y limita el dominio de conocimiento del modelo. Adicionalmente este también demuestra un patrón que se verá a través de todo el system prompt, el cual se itera múltiples veces la necesidad indispensable de utilizar las tools dadas.

You are a financial data agent. You answer questions about companies and macroeconomic indicators using ONLY the results from your available tools. Never invent, estimate, or recall data from memory.

Reglas Obligatorias (Strict Rules)

Después de definir el rol, el prompt incluye un conjunto de reglas estrictas que controlan el comportamiento del agente:

STRICT RULES:

- 1. Output to the user using ONLY the tools provided. Do not attempt to answer questions without them.*
- 2. ALWAYS call a tool before giving a FINAL answer. FINAL without a prior tool call is forbidden.*
- 3. Choose the most appropriate tool for the user's question using the DATA SOURCE ROUTING rules below.*
- 4. Never call the same tool more than once per request.*
- 5. Once you receive a tool result, immediately output FINAL using only that result.*

Estamos repitiendo el patron visto anteriormente, y es super importante que el agente entienda su incapacidad. A traves del desarrollo del proyecto fuimos agregando reglas,

como que el modelo utilizara la misma herramienta multiples veces en un ciclo hasta que se quedara sin ciclos. El punto 5 limita que el usuario pregunte dos cosas sin relación alguna, pero si permite que el modelo utilice multiples tools para responder una sola pregunta.

Data Source Routing

La sección de Data Source Routing define reglas de enrutamiento para decidir qué herramienta debe utilizarse dependiendo del tipo de pregunta del usuario, por ejemplo:

- Preguntas sobre CETES o inflación mexicana → herramientas de Banxico.
- Preguntas sobre tipos de cambio → herramienta de exchange rates por yfinance.

Ejemplo:

CETES, TIE, UDIs → use the specific Banxico tool.

Esta capa actúa como una política de selección automática de herramientas (tool routing policy), ayudando al modelo a elegir correctamente la fuente de datos.

Formato de Respuesta

El prompt también restringe el formato exacto de salida que puede generar el modelo.

*RESPONSE FORMAT (choose exactly one per turn):
ACTION: tool_name(ARGUMENT)
FINAL: <your answer using only the tool result>*

Esto simplifica el proceso de parsing dentro del agente y permite que el sistema interprete automáticamente cuándo debe ejecutarse una herramienta y cuándo debe finalizar la conversación.

Definición de Herramientas (Available Tools)

La sección más extensa del prompt contiene la definición detallada de cada herramienta disponible.

- Cada herramienta incluye:

- Nombre y firma de la función.
- Descripción de uso.
- Casos en los que debe utilizarse.
- Formato esperado de retorno.
- Ejemplos de invocación.
- Ejemplo dado en el mismo código:

```
### tool_name(arg1, arg2, ...)
```

Description: What the tool does and when to use it.

Returns: What the tool output looks like.

Example call: ACTION: tool_name(ARGV1)

```
### get_udis(date)
```

Description: Returns the value of UDIs (Unidades de Inversión) in MXN from Banxico.

If no date is provided, returns the most recent observation.

Returns: "The value of UDIs in Mexico (<label>) is <value> MXN as of <DD/MM/YYYY>."

Example calls: ACTION: get_udis()

ACTION: get_udis(2024-06-01)

Esta estructura funciona como una forma de documentación interna para el modelo, permitiéndole comprender: qué hace cada herramienta, cuándo utilizarla y cómo construir correctamente los argumentos. Además, también sigue un la referencia al lenguaje de markdown al hacer `###`, implicando semi título, o una área importante del texto.

Ejemplos de Uso (Few-Shot Prompting)

Después de las herramientas, a pesar de que en las herramientas ya se le han dado las instrucciones de cómo hacerlo con ejemplos, se decidió agregar una segunda área para demostrar casos ejemplos:

User: What is the current UDI value?
ACTION: get_udis()
After tool result: FINAL: The value of UDIs in Mexico (most recent) is 8.2341 MXN as of 30/04/2025.

De esta manera aplicamos el mismo patrón inicial, utilizando herramientas, y utilizando este sistema iterativo para poder resolver problemas complejos. Adicionalmente estos ejemplos implementan una técnica conocida como *few-shot prompting*, donde el modelo aprende patrones de comportamiento observando ejemplos previos dentro del prompt.

Ejemplos Incorrectos y Restricciones

Finalmente, el prompt incluye ejemplos explícitos de comportamientos prohibidos, reforzando el comportamiento y quitando errores comunes, dentro de nuestra iteración de desarrollo. Esto fue implementado por algunos errores, que surgen, cosas que no se podían agregar anteriormente, y se agregaron como lógica al final del sistema prompt.

INCORRECT (never do this):
ACTION: get_price_on_date(BIMBOA) <- missing .MX suffix for BMV-listed stock
ACTION: get_price_on_date(CUERVO) <- missing .MX suffix; correct is CUERVO.MX

4.4 Herramientas

Herramienta	Descripción
get_price_on_date	Obtiene el precio de cierre de una acción en una fecha específica o el precio más reciente disponible.

get_company_profile	Recupera información general de una empresa, incluyendo sector, industria y descripción del negocio.
min_variance_portfolio	Calcula una distribución de portafolio que minimiza el riesgo total utilizando varianza mínima.
max_sharpe_portfolio	Genera un portafolio optimizado para maximizar el ratio de Sharpe y el retorno ajustado al riesgo.
min_target_semivariance_portfolio	Minimiza el riesgo de desempeño negativo respecto al índice S&P 500.
get_cetes_N	Obtiene la tasa CETES a N días desde Banxico.
get_tie_N	Recupera la TIIE a N días desde Banxico.
get_target_interest_rate_mexico	Obtiene la tasa objetivo del Banco de México.
get_mensual_inflation_mexico	Recupera la inflación mensual de México.
get_inflation_mexico	Recupera la inflación anual de México.
get_udis	Obtiene el valor actual de las UDIs en pesos mexicanos.
get_exchange_rate	Consulta el tipo de cambio entre dos monedas utilizando datos de mercado.
get_news_sentiment	Analiza el sentimiento de noticias financieras usando FinBERT.
calculate_inflation_impact	Calcula la pérdida del valor del dinero a lo largo de cierta cantidad de meses utilizando la tasa de inflación reportada por Banxico.

get_fundamental_analysis	Realiza un análisis fundamental de una compañía para dar al usuario una recomendación de compra/venta de la acción.
respond_to_greeting	Responde saludos generales del usuario.
respond_no_available_tool	Responde cuando el agente no posee una herramienta adecuada para la solicitud.

5. Resultados

<i>Build a portfolio that minimizes losses relative to the S&P500 using these 5 stocks: Apple, Microsoft, Google, Amazon and Facebook.</i>
<i>To minimize downside risk relative to the S&P 500, the optimal weights across AAPL, MSFT, GOOGL, AMZN, and META are AAPL: 40.16%, MSFT: 27.95%, GOOGL: 23.40%, AMZN: 8.49%, and META: 0%, with an expected annual return of 25.58% and annualized volatility of 21.76%.</i>
<pre>[{ "step": 1, "type": "error", "tool": "min_target_semivariance_portfolio", "args": ["AAPL", "MSFT", "GOOGL", "AMZN", "FB"], "error": "Unable to coerce to DataFrame, shape must be (218, 5): given (500, 1)" }, { "step": 2, "type": "unknown",</pre>

```

    "raw": "It seems that the tool is having trouble processing the request. Let me try
again with a different approach."
  },
  {
    "step": 3,
    "type": "action",
    "tool": "min_target_semivariance_portfolio",
    "args": [
      "AAPL",
      "MSFT",
      "GOOGL",
      "AMZN",
      "META"
    ],
    "result": "Optimal weights for minimum target semivariance portfolio:\n{'AAPL':
np.float64(0.4016), 'MSFT': np.float64(0.2795), 'GOOGL': np.float64(0.234), 'AMZN':
np.float64(0.0849), 'META': np.float64(0.0)}\nExpected annual return:
25.58%\nAnnualized volatility: 21.76%"
  },
  {
    "step": 4,
    "type": "final",
    "content": "To minimize downside risk relative to the S&P 500, the optimal
weights across AAPL, MSFT, GOOGL, AMZN, and META are AAPL: 40.16%,
MSFT: 27.95%, GOOGL: 23.40%, AMZN: 8.49%, and META: 0%, with an expected
annual return of 25.58% and annualized volatility of 21.76%."
  }
]

```

Es importante este ejemplo, porque a pesar de que la pregunta está mal planteada, podemos ver que el modelo interpreta que cometió un error en cuanto al nombre que busco, y lo intenta de nuevo, usando el nuevo nombre ticker que utiliza facebook en el mercado (META).

Otro ejemplo sería el sentiment analysis:

```
What is the market sentiment around AMZN right now?
```

*Sentiment analysis for Amazon.com, Inc. (AMZN) across 10 recent articles:
Composite score: +0.1791 (POSITIVE).*

Top influencing headlines:

[POSITIVE 92%] Jim Cramer Says Nvidia Is 'Tough To Beat' On Corning Deal, Points Out Google, Amazon Still Have An 'Opponent With Multiple Legs Up' (Benzinga) --- [POSITIVE 78%] 3 Top Stocks to Buy in May (Motley Fool) --- [NEUTRAL 79%]

The \$1 trillion club's new members are powering the AI boom: Chart of the Day (Yahoo Finance) ---

[NEUTRAL 52%] Iran May Have Found a New Way to Threaten Trump's Economy — And It's Not Oil (24/7 Wall St.) ---

[NEUTRAL 83%] 2 Top Stocks Long-Term Investors Should Buy in May (Motley Fool)

5.2 Areas de mejora

Agregar una tool que pueda llamar a alguna librería o API, en la que se le mande el nombre coloquial de una empresa y regrese el ticker del stock deseado, ya que existen algunos casos con compañías poco conocidas o que cotizan en mercados diferentes al estadounidense, donde el LLM no es capaz de descifrar el ticker correcto. Sin embargo, sí es capaz de hacerlo para las compañías más grandes, incluso para empresas importantes del mercado mexicano como Grupo Bimbo, Becle, entre otras. Aun así, esto sería un extra, ya que evidentemente, al tratarse de una herramienta financiera, esto podría denominarse conocimiento común de la persona que la está utilizando. Por ende, ingresarían por default el nombre apropiado que quiere que el LLM busque.

Una segunda área de mejora relevante es la gestión de errores a nivel del agente. Actualmente, cuando el LLM produce una respuesta con formato inválido (ni ACTION: ni FINAL:), el parser la clasifica como tipo unknown y el agente simplemente intenta un ciclo adicional. Si bien esto funciona en la mayoría de los casos, no existe una lógica de recuperación diferenciada: el modelo no recibe retroalimentación explícita sobre qué aspecto del formato fue incorrecto, lo que puede propiciar que repita el mismo error en ciclos subsecuentes hasta agotar el límite de pasos.

Una mejora concreta sería que el parser construya un mensaje de error estructurado que especifique el tipo de falla detectada: formato inesperado, herramienta inexistente, argumentos mal contruidos, entre otros, y lo reinyecte al historial como observación del

sistema. Esto permitiría al LLM corregir su comportamiento de manera más informada en el siguiente ciclo, reduciendo el número de pasos desperdiciados y aumentando la robustez del agente ante respuestas malformadas.

Una tercera área de oportunidad corresponde a la extensión de la cobertura de datos macroeconómicos. En su estado actual, el agente se especializa en indicadores mexicanos obtenidos vía Banxico, lo cual restringe su utilidad a análisis centrados en el mercado doméstico. Incorporar fuentes de datos como FRED (Federal Reserve Economic Data) o la API del Banco Mundial permitiría al agente responder preguntas sobre tasas de interés de la Reserva Federal, indicadores de crecimiento de economías emergentes o comparativas entre tasas de política monetaria de distintos bancos centrales. Esta expansión sería coherente con el perfil de un agente financiero de uso profesional, donde el análisis raramente se circunscribe a una sola economía, particularmente en el contexto de portafolios con exposición internacional. Sin embargo, muchas de las api son de pago y no estaba planeado tener ese tipo de gastos en el alcance de este proyecto.

6. Conclusiones

El presente proyecto demostró que es posible construir un agente financiero conversacional funcional y verificable sin recurrir al reentrenamiento del modelo subyacente. Mediante la combinación del patrón ReAct con un sistema de prompt engineering cuidadosamente diseñado, se logró que el modelo Llama 3.3 70B Instruct opere de forma estructurada, consultando herramientas externas antes de emitir cualquier respuesta y absteniéndose de generar información desde memoria interna. Este enfoque resulta especialmente valioso en el dominio financiero, donde la alucinación de datos, precios, tasas, rendimientos, puede derivar en decisiones con consecuencias reales.

La arquitectura desarrollada evidenció que la calidad del system prompt es un determinante crítico del comportamiento del agente. La inclusión de reglas estrictas, enrutamiento explícito de herramientas, definición estructurada de cada función y ejemplos de few-shot prompting redujo de forma sustancial los errores de formato y los intentos incorrectos de invocación de herramientas. La iteración sobre el prompt durante el desarrollo del proyecto, agregando restricciones conforme surgían comportamientos no deseados, ilustra que el prompt engineering es un proceso empírico e incremental, no una tarea puntual.

Desde una perspectiva técnica, la integración de herramientas como yfinance para datos bursátiles, la API de Banxico para indicadores macroeconómicos mexicanos, scipy para optimización de portafolios y FinBERT para análisis de sentimiento, demostró que un agente basado en LLM puede actuar como un orquestador efectivo sobre fuentes de datos especializadas. Cada herramienta aporta una capacidad específica que el modelo por sí solo no podría ofrecer con la precisión requerida, y la suma de estas capacidades resulta en un sistema con un alcance analítico considerablemente más amplio que el de cualquiera de sus componentes de forma aislada.

Finalmente, el proyecto deja como aprendizaje central que los LLMs de gran escala, cuando se instrumentan correctamente, son capaces de razonar sobre problemas financieros complejos de manera coherente y reproducible. Sin embargo, el verdadero valor no reside en el conocimiento estático del modelo, sino en su capacidad de razonamiento iterativo acoplado a datos en tiempo real.

7. Referencias

- Ayar Labs. (2024, November 15). *Parameters (AI/ML)*. <https://ayarlabs.com/glossary/parameters-ai-ml/>
- Bergmann, D. (2025, November 17). Instruction tuning. IBM. <https://www.ibm.com/think/topics/instruction-tuning>
- Bergmann, D. (2026, April 1). ReAct Agents. IBM. <https://www.ibm.com/think/topics/react-agent>
- Chawla, A. (2026, January 17). *Implementing ReACT Agentic Pattern from scratch*. Daily Dose of Data Science. <https://www.dailydoseofds.com/ai-agents-crash-course-part-10-with-implementation/>
- Field, H. (2025, July 7). xAI updated Grok to be more 'politically incorrect.' *The Verge*. <https://www.theverge.com/ai-artificial-intelligence/699788/xai-updated-grok-to-be-more-politically-incorrect>
- Gadesha, V. (2025, November 17). Prompt engineering techniques. IBM. <https://www.ibm.com/think/topics/prompt-engineering-techniques>
- Gadesha, V. (2026, January 5). Prompt Engineering Guide. IBM. <https://www.ibm.com/think/prompt-engineering>
- GeeksforGeeks. (2026a, April 1). *Self attention in NLP*. GeeksforGeeks. <https://www.geeksforgeeks.org/nlp/self-attention-in-nlp/>
- GeeksforGeeks. (2026b, May 2). *Large Language Model (LLM)*. GeeksforGeeks. <https://www.geeksforgeeks.org/artificial-intelligence/large-language-model-llm/>

- Google Cloud. (2026, May 4). *Prompt engineering: overview and guide*. Retrieved May 12, 2026, from <https://cloud.google.com/discover/what-is-prompt-engineering>
- *meta-llama/Llama-3.3-70B-Instruct* · Hugging Face. (2024, December 6). <https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct>
- Nieto, A. (2025, August 7). LLM Pre-Training and Custom LLMs. *Databricks*. <https://www.databricks.com/blog/llm-pre-training-and-custom-llms>
- ProsusAI. (n.d.). *GitHub - ProsusAI/finBERT: Financial Sentiment Analysis with BERT*. GitHub. <https://github.com/ProsusAI/finBERT>
- Siddiqui, K. (2025, November 18). From pilots to production: Why AI in finance often stalls—and how to fix it. *IBM*. <https://www.ibm.com/think/insights/why-ai-finance-fails>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017, June 12). *Attention is all you need*. arXiv.org. <https://arxiv.org/abs/1706.03762>

8. Repositorio en GitHub

https://github.com/luisejdm/Proyecto3_Deep_Learning

9. Demostración en Hugging Face

https://huggingface.co/spaces/ITESO-Deep-Learning/Financial_Agent

10. Video demostrativo

<https://youtu.be/y02GJOJglfA>